

Tarea Java por parejas

José Manuel Sánchez Álvarez

25-09-2025

Índice de contenidos

Hoja de ejercicios — GeoNotesTeaching (Java 21)	1
Cómo empezar (5)	1
Bloque A — Fundamentos y calentamiento (20–30)	1
Bloque B — Jerarquía <i>sealed</i> y switch moderno (25–35)	2
Bloque C — <i>Text Blocks</i> y exportación (20–25)	2
Bloque D — Colecciones y orden (25–30)	3
Bloque E — <i>Pattern Matching + Record Patterns</i> (Java 21) (20–30)	3
Bloque F — Errores y robustez (15–20)	4
Bloque G — Extensión opcional (si hay tiempo)	4
Entrega sugerida	4
Rúbrica (10 pts)	5
Apéndice — Snippets útiles	5

Hoja de ejercicios — GeoNotesTeaching (Java 21)

Cómo empezar (5)

1. Abre el proyecto en IntelliJ (File → Open... → `geonotes-teaching-java21/`).
2. Asegúrate de **JDK 21**.
3. Ejecuta:
 - CLI: Tasks > application > run
 - Ejemplos: Tasks > application > examples

Bloque A — Fundamentos y calentamiento (20–30)

A1. Validación y excepciones

Objetivo: reforzar validación clásica y mensajes claros.

- En `Note`, añade validaciones adicionales:
 - `title` → mínimo 3 caracteres.
 - `content` → recorta con `trim()`; si queda vacío, usa "-".
- Maneja la excepción en el menú (ya lo hace) y muestra un mensaje útil. **Pista:** usa el *compact constructor* del `record`.

A2. Equals/HashCode vs. Records (conceptual)

Objetivo: entender qué genera un record.

- Crea una clase `LegacyPoint` (clásica, *no record*) con `double lat`, `lon`, `equals`, `hashCode` y `toString` manuales.
 - Compara su uso con `GeoPoint`. **Entrega:** breve comentario en el código o README: ¿qué ventajas / cuándo *no* usar record?
-

Bloque B — Jerarquía *sealed* y `switch` moderno (25–35)

B1. Nuevo subtipo: `Video`

Objetivo: ampliar jerarquía sellada.

- Crea `public record Video(String url, int width, int height, int seconds) implements Attachment`.
- Actualiza `Attachment` (`permits ...`) para incluir `Video`.
- Añade soporte en `Describe.describeAttachment`:

```
case Video v when v.seconds() > 120 -> " Video largo";
case Video v -> " Video";
```

- **Exhaustividad:** comprueba que el `switch` obliga a cubrir `Video`.

B2. Formato corto vs. largo en `switch`

Objetivo: usar `yield` con bloques.

- Cambia alguna rama de `Describe` a bloque:

```
case Audio a when a.duration() > 300 -> {
    var mins = a.duration() / 60;
    yield " Audio (" + mins + " min)";
}
```

- Asegúrate de compilar y probar.
-

Bloque C — *Text Blocks* y exportación (20–25)

C1. Export JSON pretty

Objetivo: mejorar legibilidad del JSON.

- En `Timeline.Render.export()`, ajusta el *text block* para alinear y sangrar mejor.
- Escapa comillas del `content` si hiciera falta (p. ej., `replace("\\"", "\\\"")` antes de `formatted`).

C2. Export Markdown (extra)

Objetivo: practicar *text blocks*.

- Crea `MarkdownExporter` (implementa `Exporter`) que genere:

```
# GeoNotes
- [ID 1] Título - (lat, lon) - YYYY-MM-DD
```

- Muestra su salida desde la CLI (añade opción 6: “Exportar Markdown”).
-

Bloque D — Colecciones y orden (25–30)

D1. Orden por fecha y límite

Objetivo: practicar Streams y Comparator.

- Añade método en Timeline:

```
public java.util.List<Note> latest(int n)
```

que devuelva las *n* notas más recientes (por `createdAt` descendente).

- Añade opción en CLI: “Listar últimas N”.

D2. Búsqueda con varios criterios

Objetivo: filtros encadenados.

- En CLI, añade una opción “Buscar avanzada”:
 - Por rango de lat/lon (ej.: lat entre A–B).
 - Por palabra clave en `title` o `content`.
 - Reutiliza `Match.isInArea` o crea un método auxiliar.
-

Bloque E — *Pattern Matching* + *Record Patterns* (Java 21) (20–30)

El proyecto docente usa un enfoque clásico en `Match`, pero ahora practicaremos lo nuevo.

E1. `instanceof` con patrón

Objetivo: simplificar *casting*.

- En `Describe` añade un método:

```
static int mediaPixels(Object o)
```

que:

- Si es `Photo p`, devuelva `p.width() * p.height()`.
 - Si es `Video v`, devuelva `v.width() * v.height()`.
 - Si no, 0.
- Implementa con `if (o instanceof Photo p) { ... }`.

E2. *Record patterns* en `if` o `switch`

Objetivo: desestructurar con patrón.

- Crea método en `Match`:

```
static String where(GeoPoint p)
```

que use:

```
return switch (p) {
    case GeoPoint(double lat, double lon) when lat == 0 && lon == 0 -> "ORIGIN";
    case GeoPoint(double lat, double lon) when lat == 0 -> "Equator";
    case GeoPoint(double lat, double lon) when lon == 0 -> "Greenwich";
    case GeoPoint(double lat, double lon) -> "(" + lat + "," + lon + ")";
};
```

- Añade opción CLI para consultar `where`.

Bloque F — Errores y robustez (15–20)

F1. Manejo de `InputMismatch/NumberFormatException`

Objetivo: entradas seguras.

- Asegura que **todas** las lecturas de números usan `Double.parseDouble(scanner.nextLine())` y están en `try/catch` con mensajes claros (ya está iniciado en `GeoNotes`).

F2. Comprobaciones nulas

Objetivo: práctica “clásica” (sin null-safety de Kotlin).

- Si `label` en `Link` es nulo/vacío, muestra la `url` al exportar (ya implementado en `Describe`; revisa consistencia en exportadores).

Bloque G — Extensión opcional (si hay tiempo)

G1. Vista invertida (java 21, `Sequenced`)

Objetivo: mostrar la API moderna sin cambiar el enfoque clásico.

- Sustituye el `Map<Long, Note>` interno por `SequencedMap<Long,Note>` (con `LinkedHashMap`).
- Añade método:

```
public java.util.Collection<Note> reversed() { return notes.reversed().values(); }
```

- Opción CLI: “Listar (reversed)”.

G2. Demo *virtual threads* (muy opcional)

Objetivo: idea general de Loom.

- Crea `VirtualDemo.runIO()` que lance ~50 tareas “simuladas” (sleep 200–300 ms) con:

```
try (var exec = java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) { ... }
```

- Muestra el hilo actual en cada tarea. **No** mezclar con la lógica del proyecto (solo demo).

Entrega sugerida

- Cambios en el código + pequeño README con:
 - Lista de ejercicios hechos.

- Notas sobre decisiones de diseño y Java vs Kotlin (2–4 bullets).

Rúbrica (10 ptos)

- A–B: Validación, sealed + switch/guards (4 ptos).
 - C–D: Text Blocks/Exporter, Streams/orden (3 ptos).
 - E: Pattern matching + record patterns (2 ptos).
 - F: Robustez de entradas + nulos (1 pto). *(+1 extra por G1 o G2)*
-

Apéndice — Snippets útiles

Orden por fecha:

```
var latest = timeline.getNotes()
    .values().stream()
    .sorted(java.util.Comparator.comparing(Note::createdAt).reversed())
    .limit(n)
    .toList();
```

Text Block con .formatted(...):

```
String json = """
    { "title": "%s", "content": "%s" }
    """.formatted(title, content.replace("\n", "\\n"));
```

instanceof con patrón:

```
if (obj instanceof Photo p) {
    // p ya está casteado a Photo
}
```

switch con yield:

```
String label = switch (a) {
    case Audio au when au.duration() > 600 -> {
        var min = au.duration() / 60;
        yield " Podcast (" + min + " min)";
    }
    case Audio au -> " Audio";
    default -> "Otro";
};
```
